

PROGRAMMING · BEGINNER

C Programming Fundamentals

Start with C. Learn how programs really work.



A free, practical foundation in structured programming with C.

🕒 6-8 weeks

📁 4 modules

📁 Free course

[Explore the learning experience →](#)

Practical learning. Clear progression. Work you can explain.

YOUR NEXT CAPABILITY

What will you be able to do?

Learn syntax, control flow, functions, arrays, pointers, structures and file handling through focused exercises and a small terminal project.

 Write and debug structured C programs

 Use functions, arrays, pointers and structures

 Work safely with files and memory

 Complete a command-line capstone

Who this is for

- Programming beginners
- Engineering students
- Learners preparing for C++ or embedded systems

Your starting point

- No prior programming experience required

Your progression at a glance

FROM CONCEPT TO EVIDENCE



Learn it. Apply it. Explain it.

Move from guided concepts to practical exercises and a coherent piece of work. Use the course resources to revisit difficult ideas and make your reasoning visible.

THE CURRICULUM

A clear route through the subject.

Every module builds towards practical work. The course outline below is aligned with the academy's current program catalog.

1 Programming and C Foundations

Tooling, syntax, variables and program flow.

- Set up the C toolchain
- Variables and data types
- Operators and expressions
- Input and output

2 Control Flow and Functions

Decisions, loops and reusable program structure.

- Conditions
- Loops
- Functions and scope
- Debugging fundamentals

3 Arrays, Strings and Pointers

Core memory and collection concepts.

- Arrays
- Strings
- Pointer foundations
- Pointers with arrays

4 Structures, Files and Capstone

Model data and persist it safely.

- Structures and enums
- File handling
- Memory safety review
- Terminal application capstone

FROM CONCEPT TO EVIDENCE



> Practice with purpose

Use functions, structures and file operations to create a small terminal application and explain how its data moves through memory.

BUILD SOMETHING THAT DEMONSTRATES YOUR LEARNING

Command-line record manager

Use functions, structures and file operations to create a small terminal application and explain how its data moves through memory.

**Readable C source files****Input and boundary-case tests****Memory and file-handling explanation**

Tools and working methods

C compiler

VS Code

Git

Tools are part of the learning workflow, not partner endorsements. Examples may evolve while preserving the learning outcomes.



Delivery

Self-paced online · 6–8 weeks

Confirm the current schedule, delivery arrangements and any prerequisites before enrolling.



Completion

A completion certificate is available after the required course work and a passing assessment.

Course completion is not a job, placement or funding guarantee.

Your next steps

- 1 Review the curriculum and your starting point.
- 2 Create your learner account and enroll for free.
- 3 Start learning, keep your evidence and track your progress.

[Open the course experience →](#)

Customer-preview edition. Interactive links open the local academy prototype. Confirm course availability, schedule and commercial terms with the academy.

INSIDE THE LEARNING EXPERIENCE

Compute a sensor average without losing precision

A temperature logger stores integer readings, but the average may be fractional. You need a reusable function that keeps the input unchanged and handles an empty reading set.

Understand the concept

An array argument does not carry its element count into a C function. Pass the count separately and use `size_t` for indexing. Accumulate into a double so the final division is floating-point rather than integer division. Check the count before dividing; the caller must distinguish missing data from an actual zero reading.

Worked example

```
int readings[] = {21, 24, 26};
size_t count = sizeof readings / sizeof readings[0];
double total = 0.0;
for (size_t i = 0; i < count; ++i) {
    total += readings[i];
}
if (count > 0) printf("%.2f\n", total / count);
```

Interpret the result

The three readings total 71. Dividing by 3 produces 23.67 when formatted to two decimal places. A loop condition of `i <= count` would access an element outside the array.

Knowledge check

Why is `total` initialized as `double` rather than `int`?

Explanation: It preserves a fractional result during division and avoids integer truncation in the averaging expression.

YOUR PRACTICAL ASSIGNMENT

Build evidence, not just familiarity.

Compute a sensor average without losing precision



Your task

- Extract the calculation into a function with an explicit empty-input policy.
- Add minimum and maximum readings without modifying the array.
- Read values from a local text fixture and report malformed lines.



Submission review criteria

- Correct average for positive and negative readings
- No division by zero or out-of-bounds access
- Tests for one reading, repeated readings and no readings

Submit

A working artifact or analysis, a short explanation of your decisions, and evidence that addresses each review criterion.

Reflect

Describe one failed approach, how you diagnosed it, and what you would test next. Identify limitations instead of hiding them.

Your project connection

Use functions, structures and file operations to create a small terminal application and explain how its data moves through memory.

Use the sample assignment as a stepping stone toward the course project, not as a substitute for the full curriculum.

[Explore the worked lesson](#) →

Proposed teaching sample for curriculum and UX review. Full lesson production, instructor review and delivery scheduling remain separate work.